



南京理工大学

NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

学士学位论文

芯片缺陷检测算法设计及实现

王鹏

922106840141

指导教师

吕建勇

副教授

学生学院

计算机科学与工程学院

专业

智能科学与技术

研究方向

目标检测

论文提交时间

2026年5月

学 士 学 位 论 文

芯片缺陷检测算法设计及实现

作 者：王鹏

指导教师：吕建勇 副教授

南 京 理 工 大 学

2026 年 5 月

Bachelor Dissertation

Chip Defect Detection Algorithm Design and Implementation

By

Peng Wang

Supervised by Prof. Jianyong Lv

Nanjing University of Science & Technology

May, 2026

声 明

本学位论文是我在导师的指导下取得的研究成果，尽我所知，在本学位论文中，除了加以标注和致谢的部分外，不包含其他人已经发表或公布过的研究成果，也不包含我为获得任何教育机构的学位或学历而使用过的材料。与我一同工作的同事对本学位论文做出的贡献均已在论文中作了明确的说明。

学生签名：_____ 年 月 日

学位论文使用授权声明

南京理工大学有权保存本学位论文的电子和纸质文档，可以借阅或上网公布本学位论文的部分或全部内容，可以向有关部门或机构送交并授权其保存、借阅或上网公布本学位论文的部分或全部内容。对于保密论文，按保密的有关规定和程序处理。

学生签名：_____ 年 月 日

摘要

随着集成电路产业的迅猛发展，芯片制造工艺日益精进，其表面质量检测在封装与测试环节中扮演着至关重要的角色。为满足工业现场对检测系统轻量化、高实时性以及低部署成本的要求，本课题基于机器视觉理论，设计并实现了一套高效的芯片表面缺陷自动检测系统。

针对工业环境下图像采集易受光照波动和噪声干扰的问题，系统在图像预处理阶段引入高斯滤波与 Retinex 图像增强算法，在有效抑制背景噪声与光照不均的同时，最大限度地保留了芯片的边缘细节特征。在目标定位与图像配准阶段，本系统采用图像金字塔与由粗到精的搜索策略，实现了高效且精确的模板匹配。针对匹配过程中残留的微小对齐误差，采用局部均值差分 and 连通域分析进行像素级校正与补偿；最后，通过连通域分析精确提取疑似缺陷区域，并结合形态学特征分类策略完成最终缺陷的判定与筛选。在系统工程实现中综合运用了多线程并行计算、多尺度处理等优化策略，并遵循了界面与核心算法解耦的设计架构，显著提升了系统的运算效率与稳定性。

本系统采用 C++ 语言联合 Qt 框架实现，实验结果表明，该系统能够快速、稳定地完成芯片表面缺陷的自动化检测任务，具备较强的工程实用价值与应用前景。

关键词：芯片缺陷检测，机器视觉，模板匹配，连通域分析，大津法

Abstract

With the rapid development of the integrated circuit industry and the continuous advancement of chip manufacturing processes, surface quality inspection plays a crucial role in the packaging and testing stages. To meet the requirements of industrial environments for lightweight, high real-time performance, and low deployment costs for inspection systems, this project designs and implements a highly efficient automatic chip surface defect detection system based on machine vision theory.

Addressing the issue of image acquisition in industrial environments being susceptible to illumination fluctuations and noise interference, the system incorporates Gaussian filtering and Retinex image enhancement algorithms in the image preprocessing stage. This effectively suppresses background noise and uneven illumination while preserving the chip's edge details to the maximum extent. In the target localization and image registration stage, the system employs an image pyramid and a coarse-to-fine search strategy to achieve efficient and accurate template matching. For minor alignment errors remaining during the matching process, local mean difference and connected component analysis are used for pixel-level correction and compensation. Finally, connected component analysis is used to accurately extract suspected defect areas, and a morphological feature classification strategy is combined to complete the final defect determination and screening. The system implementation comprehensively utilizes optimization strategies such as multi-threaded parallel computing and multi-scale processing, and follows a design architecture that decouples the interface from the core algorithm, significantly improving the system's computational efficiency and stability.

This system is implemented using C++ and the Qt framework. Experimental results show that the system can quickly and stably complete the automated detection task of chip surface defects, possessing strong engineering practical value and application prospects.

Keywords: Chip defect detection, machine vision, template matching, Connectivity analysis, Otsu method

目录

摘要.....I

Abstract II

目录.....III

1 绪论 1

 1.1 研究背景 1

 1.2 国内外研究现状 2

 1.3 研究目的及意义 3

2 理论基础及技术支持.....4

 2.1 概述 4

 2.2 图像处理底层数学模型 4

 2.3 高斯平滑滤波 5

 2.4 大津法 6

 2.5 连通域分析 6

3 系统需求分析.....8

 3.1 需求分析 8

 3.1.1 设计目标 8

 3.1.2 设计原则 9

 3.2 系统可行性分析 9

 3.2.1 技术可行性 9

 3.2.2 经济可行性 10

 3.2.3 操作可行性 10

 3.3 运行需求 10

4 系统总体设计..... 12

 4.1 系统整体架构设计 12

 4.2 系统功能模块设计 13

 4.3 系统核心类与运行机制设计 14

 4.3.1 用户交互与调度机制 14

 4.3.2 核心算法与数据处理机制 14

 4.4 系统界面结构设计 15

5 系统详细设计及编码 17

 5.1 文件导入 17

 5.1.1 功能概述 17

5.1.2 图像导入与渲染流程	17
5.1.3 界面展示与检测流程	18
5.2 模板匹配	19
5.2.1 双边滤波	19
5.2.2 Retinex 增强	20
5.2.3 图像金字塔	23
5.3 特征提取	24
5.3.1 阈值分割	24
5.3.3 连通域分析	25
5.3.3 图像差分	27
5.4 缺陷检测	28
5.4.1 特征分类	28
5.4.2 全局分析	29
5.4.3 可视化结果输出	30
5.5 导出结果和系统测试	30
6 总结与展望	32
致谢	33
参考文献	34

1 绪论

1.1 研究背景

现代信息技术的基础离不开集成电路与半导体产业，而在芯片的制造与封装等环节中，像硅片表面的划痕、污渍，还有引脚变形等一系列微小缺陷，会把芯片的良率以及整个系统的可靠性给直接影响到；所以说，在半导体生产线上，缺陷检测早就变成了一道不可或缺的关键工序。另外，随着芯片制程工艺得到了不断的精进，对产能的要求也相应提高了，以前那种靠人拿着显微镜去检查的传统方式，因为检测效率偏低，加上人容易受主观疲劳影响，已经没法把现代工业的生产需求给完全满足了；这时候，基于机器视觉的自动光学检测技术，凭借着不需要接触、精度很高并且能实现自动化的特点，慢慢把人工给取代了，成为了目前控制芯片质量的一个主流手段。

就目前来看，不管是工业界还是学术界，在搭建视觉检测系统的时候，普遍会去依赖那些成熟的商业视觉软件，或者是像 OpenCV 这种大型的开源计算机视觉库；虽说这种开发模式能把系统的研发周期给有效缩短，但是在实际的工程落地过程当中，也暴露出了一些局限性。一方面，大型开源视觉库为了把不同领域的通用性都照顾到，在内部封装了极其庞杂的算法模块，这就导致了它的依赖库体积非常大，如果把它用到资源有限的边缘计算设备或者轻量化的嵌入式系统上，往往会出现内存占用偏高，以及运行过于冗余的问题；另一方面，过度去依赖这种已经封装好的算法“黑盒”，会让开发者很难针对那些特定的芯片缺陷特征，去进行底层算子级别的精细化内存调度和优化，这就或多或少地把检测软件核心逻辑的自主可控性给限制住了。

为了把前面提到的这些工程痛点给解决掉，去开发出一款足够轻量化、底层逻辑透明并且能实现自主可控的芯片缺陷检测系统，就具有了比较强的实际意义；因此，本课题没有去采用那种直接调用第三方通用视觉库的常规方案，而是从图像处理的底层数学原理出发，利用 C++ 语言，再结合上自己定义的底层矩阵数据结构，把图像处理核心模块的开发给独立完成了。整个系统通过自己去实现高斯平滑、Sobel 边缘提取、拉普拉斯锐化等关键的算法，同时结合着 Qt 框架把前端交互给构建了起来，目的就是为了搭建出一套底层逻辑很清晰、运行起来效率很高，并且完全摆脱了外部视觉库依赖的轻量级检测平台；这样一来，这不仅是对机器视觉核心算法在工程化实现上的一次实践探索，也为工业现场那些受限环境下的缺陷检测软件开发，提供到了一种切实可行的技术思路。

1.2 国内外研究现状

半导体制造与封装流程中，芯片表面缺陷的自动筛查直接决定了产线良率。国外相关课题开展较早，起初多以传统图像处理算法来定位特征。比如 Shankar 等人^[1]较早验证了机器视觉在晶圆自动化检测中的可行性；Tsai 等人^[2]利用各向异性扩散算法，成功提取出了复杂纹理下太阳能硅片的微裂纹。这类基于空域或频域计算的方法，由于数学推导明确，工程运行相对稳定。当硬件算力逐渐充裕后，国外不少研究团队开始转向深度学习体系，尝试借助卷积神经网络去应对那些非结构化、特征相对复杂的工业缺陷检测任务^[3]。

国内在电子元器件缺陷检测方面的起步虽稍晚，但目前基本形成了深度学习与传统视觉算法并行发展的态势。在神经网络分支，许多学者结合具体的车间环境对模型进行了针对性优化：像李可等^[3]改进 U-Net 结构去查验 X 射线下的焊缝气泡；朱红艳等^[5]和孙志超等^[6]分别引入可变形卷积与多尺度特征融合，以此拉高复杂 PCB 板上微小缺陷的识别精度。不过，受制于深度模型极高的算力消耗以及对样本集的高度依赖，加上在工业现场容易暴露出的“黑盒”不可控属性，传统图像处理技术依然保留着明确的落地空间。温悦欣^[7]专门给 PCB 裸板开发了一套视觉提取算法；张立国等^[8]与张定恒等^[9]分别搭建了针对电路板及焊盘的检测机制；巩玉奇等^[10]通过修改中值滤波增强了玻璃缺陷的轮廓特征；汪海涛^[11]还探索了将 GMR 传感器引入检测环节的新方案。从宏观技术应用层面看，杨思念等^[12]在综述中指明机器视觉取代人工目检已成必然；Hanlin 等^[13]的分区增强匹配算法提升了局部特征配准的鲁棒性；金贺楠^[14]针对 SOP 芯片引脚的几何规则给出了一套工程级检测方案；王平与白秀玲^[15]在改良模板匹配策略后，进一步兼顾了缺陷定位的精度与实时性。从上述文献不难看出，面向特定的车间检测任务去定向优化基础图像算子，依然是平衡运行速度与检测精度的有效途径。

为了适应工业现场对处理延迟和计算资源的苛刻限制，国内还有一部分课题尝试将视觉算法下放至异构硬件层，借此规避操作系统的调度开销。利用 FPGA 或者 ZYNQ 平台去执行图像采集与算子加速是非常主流的探索方向。冯天桥等^[16]与阳斌等^[17]先后研制出基于 FPGA 的运动目标采集与检测板卡；李庆春等^[18]测试了 ZYNQ 平台的远程推流能力；贺聪等^[19]更是直接利用硬件逻辑语言实现了自适应直方图均衡化。在处理软硬件结合与工程部署的问题时，核心算法如何与前端界面打配合也受到了关注。例如宁效龙等^[20]依托 Zynq 搭配 Qt 框架，完成了一套带有前端交互的边缘检测系统，验证了 Qt 在工业视觉跨平台显示上的可靠性。

纵观上述研究进展，虽然深度神经网络与硬件加速是当前视觉领域的热点，但普通产线在部署检测软件时，往往更加看重系统的轻量化程度以及核心逻辑的自主可控。如

果在工程中全盘打包体积庞大的第三方开源视觉库，放入算力受限的主机内极易引发程序臃肿，且不利于后续的跨设备移植。基于这种考量，本课题确定了一条兼顾效率与可控性的开发路线：不再过度依赖外部冗杂的高层接口，而是通过代码重构图像平滑、特征提取等关键算子的底层逻辑，并结合 Qt 框架完成前端交互的构建。这种将核心算法与界面解耦的模式，既能保障软件的执行效率，也契合了工业级检测软件对轻量化部署的实际期许。

1.3 研究目的及意义

半导体芯片在生产和封装阶段，对良品率的要求极为严苛。一旦硅片表面出现划痕、污渍，或是引脚发生变形，都会严重影响集成电路的正常工作。为了防止带有物理缺陷的产品流入下一道工序，目前产线急需引入自动化的视觉检测系统，以实现更准确、更快速的次品拦截。

结合工业现场硬件算力和内存较为有限的实际情况，本项目设计并实现了一个轻量级的缺陷检测软件。在代码编写阶段，考虑到常见的第三方视觉库往往体积较大且存在计算冗余，本系统选择使用 C++ 自行构建图像矩阵的存储与处理逻辑。项目中独立完成了高斯平滑、Sobel 边缘检测以及连通域分析等基础算法的代码实现。这种处理方式不仅降低了软件对外部运行库的依赖程度，也使得图像缺陷定位过程中的数据流转更加清晰。

作为一次专业综合实践，本次毕业设计不仅包含了基础图像算法的数学原理推导，还涉及了像素数据的内存操作，并使用 Qt 框架完成了图形用户界面的搭建。整个开发过程帮助我将课堂上学到的机器视觉理论知识，真正应用到了具体的软件工程实践中，是一次非常有价值的锻炼。

总结来说，本设计所实现的视觉检测系统，在保证图像处理速度和数据透明度的前提下，较好地满足了工业现场轻量化部署的需求。项目最终跑通的软件架构和算法机制，不仅验证了理论的可行性，也为芯片表面缺陷的自动化筛查提供了一种基础的参考方案。

参考。

2 理论基础及技术支持

2.1 概述

表面缺陷检测系统主要由图像采集、数据处理和界面交互三个模块组成。考虑到工业现场计算机的算力与存储资源较为有限, 为了降低软件运行开销, 本项目并未调用体积较大的第三方视觉开源库。系统内部的图像矩阵存储与处理流程, 均结合相关的基础数学模型独立编程完成。这种设计有效减小了系统占用, 满足了车间轻量化部署的实际需求, 同时也帮助我将所学的机器视觉专业知识真正运用到了工程实践当中。

为配合该系统的开发工作, 本章将对涉及的相关图像处理理论进行重点回顾。内容首先介绍二维离散卷积的基本运算原理, 随后分别阐述高斯平滑滤波、Otsu 大津法以及连通域标记算法的具体运算机制。将这些核心算法的数学面貌梳理清楚, 能够为后续各功能模块的 C++ 代码编写与系统整体联调提供必要的理论支撑。

2.2 图像处理底层数学模型

一副图像可以定义为一个二维函数 $f(x, y)$, 其中 x 和 y 是空间(平面)坐标, 任意一对空间坐标 (x, y) 处的幅度值 f 称为图像在该坐标点的强度或灰度。当 x, y 和灰度值 f 都是有限的离散量时, 我们称该图像为数字图像。数字图像处理是指借助于数字计算机来处理数字图像。注意, 数字图像由有限数量的元素组成, 每个元素都有一定的位置和数值, 这些元素称为像素。

数字图像是一个二维的离散信号, 对数字图像做卷积操作其实就是利用卷积核(卷积模板)在图像上滑动, 将图像点上的像素灰度值与对应的卷积核上的数值相乘, 然后将所有相乘后的值相加作为卷积核中间像素对应的图像上像素的灰度值, 并最终滑动完所有图像的过程。如式(2.1)所示:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (2.1)$$

原理如图 2.1 所示。

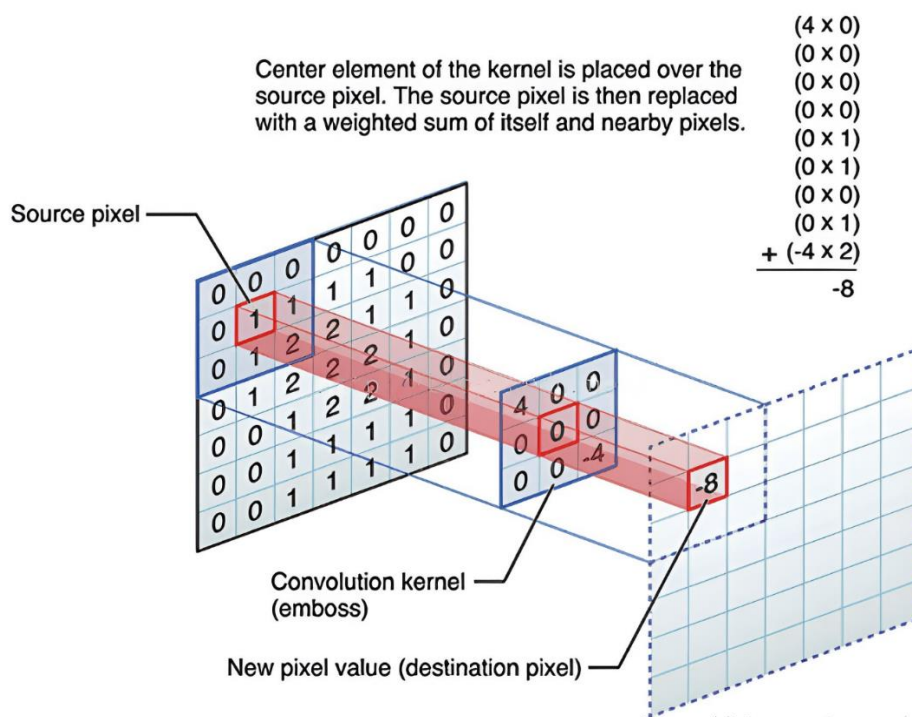


图 2.1 图像卷积

2.3 高斯平滑滤波

与赋予邻域像素相同权重的均值滤波不同，高斯滤波是一种基于正态分布的线性平滑滤波方法。其核心思想是对图像邻域内的像素进行加权平均，距离中心像素越近的点权重越大，距离越远的点权重越小。在二维连续空间中，均值为 0 的高斯函数数学表达式(2.2)：

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (2.2)$$

式中， x 与 y 代表邻域内像素点相对于中心像素点的空间坐标偏移量； σ 为高斯分布的标准差。标准差的大小直接决定了图像平滑的程度：值越大，高斯分布曲线越平缓，平滑效果越显著，但图像细节丢失也越严重；反之，值越小，图像边缘保留越好，但去噪能力相对减弱。

在实际的软件工程实现中，由于计算机处理的数字图像本质上是离散的二维矩阵，因此必须将连续的二维高斯函数进行离散化采样，以生成可用于矩阵卷积运算的数字滤波模板（即高斯核矩阵）。以构建 3×3 大小的高斯核为例，需以核中心为原点(0,0)，计算坐标从(-1,-1)到(1,1)各个位置的高斯函数值。

为了保证图像在经过高斯滤波处理后整体的灰度亮度保持恒定，避免像素值溢出，必须对生成的高斯核矩阵进行归一化处理。归一化的数学逻辑是将矩阵中的每一个元素

除以所有元素之和，使得高斯核矩阵内所有权值的总和严格等于 1。其离散化与归一化后的权值计算公式为式(2.3)：

$$K(x, y) = \frac{G(x, y)}{\sum_{i=-a}^a \sum_{j=-b}^b G(i, j)} \quad (2.3)$$

2.4 大津法

大津法本质上是一种基于聚类思想的统计算法，其核心评判准则是“最大类间方差”。算法假设存在一个阈值 t ($0 \leq t < L$, L 通常为 256 个灰度级)，将图像中的所有像素划分为背景（类 C_0 ）与目标缺陷（类 C_1 ）两部分。

设图像总像素数为 N ，灰度级为 i 的像素数为 n_i 则对应灰度级出现的概率为 $p_i = n_i/N$ 被阈值 t 划分出的两类的发生概率 ω_0 和 ω_1 计算公式见式(2.4)：

$$\omega_0 = \sum_{i=0}^t p_i, \omega_1 = \sum_{i=t+1}^{L-1} p_i = 1 - \omega_0 \quad (2.4)$$

类 C_0 与类 C_1 的灰度均值 μ_0 和 μ_1 见式(2.5)：

$$\mu_0 = \frac{\sum_{i=0}^t i \cdot p_i}{\omega_0}, \mu_1 = \frac{\sum_{i=t+1}^{L-1} i \cdot p_i}{\omega_1} \quad (2.5)$$

图像全局的灰度均值 μ_T 可表示为 $\mu_T = \omega_0 \mu_0 + \omega_1 \mu_1$ 。根据方差的定义，背景与目标之间的类间方差 σ_B^2 的计算公式推导如式(2.6)：

$$\sigma_B^2 = \omega_0 (\mu_0 - \mu_T)^2 + \omega_1 (\mu_1 - \mu_T)^2 = \omega_0 \omega_1 (\mu_0 - \mu_1)^2 \quad (2.6)$$

类间方差能够极其灵敏地反映两类数据的离散程度。当达到最大时，意味着背景与缺陷区域被错分的概率降至最低，此时对应的 t 即为所求的最佳全局二值化阈值。

2.5 连通域分析

连通域的判定依赖于像素邻域 (Neighborhood) 的拓扑概念。对于图像中的任意像素点 $p(x, y)$ ，其空间连接关系主要分为四连通 (4-Connectivity) 与八连通 (8-Connectivity)。四连通仅考虑像素点上下左右四个方向的相邻关系，其邻域集合 $N_4(p)$ 可表示为式(2.7)：

$$N_4(p) = \{(x \pm 1, y), (x, y \pm 1)\} \quad (2.7)$$

而八连通则在此基础上增加了四个对角线方向的相邻关系，其邻域集合 $N_8(p)$ 表示为式(2.8)：

$$N_8(p) = N_4(p) \cup \{(x \pm 1, y \pm 1), (x \pm 1, y \mp 1)\} \quad (2.8)$$

芯片表面的划痕和细微裂纹往往呈现倾斜或对角的分布特点。如果单纯使用四连通规则，很容易将原本连续的缺陷误判为断裂的多个部分。因此，为了保证缺陷形状的完整，本系统统一选择八连通准则进行图像遍历。在编写代码时，主要利用了两遍扫描法（Two-Pass）配合并查集来实现连通域的标记。

第一遍扫描是逐行进行的。当程序读取到前景像素时，会去检查它左侧、左上、正上和右上四个已访问过的相邻位置。如果这四个邻域都没有标记，就给当前像素分配一个新的标签序号；反之，就取这些邻域里最小的标号赋给当前像素，同时利用一维数组把这几个局部标签的等价连通关系记录下来。

经过第一轮遍历，同一个缺陷实体通常会被分配好几个不同的临时编号。为了合并这些冗余编号，算法利用并查集的路径压缩机制，找出每个等价集合最终的根节点。在进行第二遍扫描时，程序直接根据建立好的等价映射表，把图像里所有的局部标签统一替换成对应的根节点序号。通过这套处理流程，系统最终准确地将各个独立的缺陷区域划分了出来。

在完成区域标记后，系统同步进行缺陷特征的数学量化。设第 k 个连通域的像素集合为 C_k ，则该缺陷区域的面积（即包含的像素总数） $Area_k$ 的计算公式为式(2.9)：

$$Area_k = \sum_{(x,y) \in C_k} 1 \quad (2.9)$$

同时，为了在系统前端（Qt 界面）能够直观地框选出缺陷位置，系统需计算每个连通域的外接矩形边界。其左上角坐标 (X_{min}, Y_{min}) 与右下角坐标 (X_{max}, Y_{max}) 的提取逻辑为式(2.10)和式(2.11)。

$$X_{min} = \min_{(x,y) \in C_k} x, X_{max} = \max_{(x,y) \in C_k} x \quad (2.10)$$

$$Y_{min} = \min_{(x,y) \in C_k} y, Y_{max} = \max_{(x,y) \in C_k} y \quad (2.11)$$

在编写 C++ 代码时，如果使用传统的深度优先搜索（DFS）递归算法来标记连通域，当遇到面积较大的缺陷图像时，很容易出现栈溢出的问题。为了避免这种情况，本系统采用了基于并查集的两遍扫描法（Two-Pass）。该方法利用一维数组来记录和维护等价关系，不需要进行很深的递归调用，只需对图像进行两次遍历，就能分析出各个区域的连通情况。

标记完成后，程序会计算出每个独立连通区域的面积，并将其与预设的缺陷面积阈值进行大小比较，从而过滤掉那些细小的背景噪点。最后，系统把筛选出的真实缺陷坐标发送给 Qt 界面模块，以此来完成最终的图像显示与报警提示。

3 系统需求分析

开发任何软件都需要建立在明确的应用场景之上。结合半导体封装产线的实际质检需要，本章重点对即将开发的缺陷检测系统进行前期需求分析。具体工作主要包括：梳理软件的功能与非功能需求，确定系统的软硬件运行环境，并针对自行编写底层图像算法这一技术路线进行可行性论证。做好上述基础的需求调研与界定，将为后续搭建软件整体架构和编写核心代码提供确切的指导方向。

3.1 需求分析

3.1.1 设计目标

开发这个系统的主要目的，是去打造一套把图像处理、特征提取以及交互展示结合到一起的芯片缺陷检测软件，具体会实现下面几个核心的功能：

第一点，在图像数据的接入和管理上，系统能把工业相机采集到的那些常见格式图片（比如 BMP、JPG、PNG 这些）导入进来，不仅支持文件一张一张地加载，也支持批量的读取，从而确保了图像的矩阵数据能够在系统内部被妥善地组织起来并安全地传递下去；

第二点，关于图像的预处理与特征增强，提供了平滑滤波和锐化处理的手段，这能把工厂实际环境下光学采集时产生的那些随机噪点给有效地压制住，同时还会把表面划痕或者引脚变形这种细小缺陷的边缘轮廓给强化出来，进而为后面的图像分割环节准备好了质量更高的输入素材；

第三点，在自适应的缺陷分割方面，它具备依靠全局灰度统计特点来进行自适应阈值分割的能力，可以很好地克服掉实际生产线上因为打光不均匀、或者是光源亮度下降等环境因素造成的干扰，精准地把可能存在缺陷的区域从芯片正常的背景当中给剥离出来；

第四点，为了做好特征的量化和目标提取，系统构建出了一套完整的缺陷形态学处理加上连通域分析的流程，能把图片上那些极微小的粉尘噪点给精确剔除掉，准确算出来每一个真正缺陷的面积参数，并且提取出它们对应的空间位置坐标；

第五个方面，针对人机交互与参数的配置，搭建了一个直观又方便使用的图形用户界面，支持对图像进行放大缩小以及平移这些操作，还能把处理后的结果给实时地呈现出来；

第六个方面，涉及到判定结果的输出和展示，它会根据刚才算出的缺陷特征，再去结合预先设定好的公差标准，去自动判断并显示这块芯片目前的检测结果到底是合格还是不合格（OK/NG），而且还会直接在原图上用彩色的框线把缺陷的具体位置给清晰地

标定出来。

3.1.2 设计原则

为了让缺陷检测软件在工厂实地应用时，能完全满足高效、可靠以及好用的要求，系统的总体设计遵循了下面这四个基本原则：

第一点，在模块化架构方面，把整个系统的逻辑清晰地划分成了界面展示、图像解析还有核心算法处理这几个模块，各个模块之间会通过标准化的接口去交换传递数据，这样不仅能把代码的耦合度给有效地降下来，也方便了后续对算法进行迭代升级以及对系统的整体维护；

第二点，为了确保核心算法的稳定和准确，考虑到半导体硅片本身的表面容易反光，再加上现场环境里存在的各种噪声干扰，专门去优化了预处理与形态学的算法逻辑，从而保证系统在碰到不同批次和不同光照条件的检测场景时，都能把检测的精度保持在一个稳定的水平，并且严格控制住误报与漏检的情况；

第三点，因为工业流水线对检测的节奏速度有着严格的限制，为了做到高效运转和实时响应，系统在设计时把优化的重点放在了矩阵运算的耗时以及内存的调度机制上，以此来保证能够快速处理那些高分辨率的图片，并确保界面操作顺畅无卡顿，还能把最终的检测结果给实时输出出来；

第四点，在界面的设计上非常贴合一线操作工人的日常习惯，提供了直观的视觉层级和操作上的引导，把那些极具复杂性的算法逻辑都放到了后台去静默运行，前端页面只保留了那些关键并且很容易看懂的调节参数，进而把整个系统的操作门槛给大幅度降低了。

3.2 系统可行性分析

3.2.1 技术可行性

第一个方面是算法非常成熟，这个系统里用到的一些核心计算方法（像高斯平滑滤波、Sobel 边缘提取、大津法自适应阈值分割、形态学运算还有连通域分析这些）都是处理数字图片和做机器视觉的老牌经典手段，它们都有着特别扎实的数学理论做支撑，而且在工厂里做平面的缺陷检查以及提取轮廓这些事情上，已经用过几十年的时间了，早就经历过各种实际操作的反复检验，这就说明把它们应用在这里，不管是从理论上看还是从技术路线上看都是完全成型的；

第二个方面在于开发环境和框架的支持很到位，系统最底层的算法逻辑是用标准 C++ 语言去编写的，而前端那个交互界面用到了 Qt 这个能跨平台开发的框架，因为 C++ 语言在管理内存还有运行速度上有极大的优势，能顺利地把高分辨率工业图片带来的一大堆矩阵运算给处理掉，同时 Qt 框架又能提供非常稳定的图形显示部件和信息传递机

制,把这两种技术凑到一起,正是现在工业圈子里开发高性能视觉检测软件的主流做法,所以说开发技术绝对是可行的;

第三个方面体现在数据的兼容以及平台移植会很方便,它把核心的算法部分和外部的系统界面分得特别开,并且设定了十分规范的数据对接标准,这样系统不仅能顺利读懂 BMP、PNG 还有 JPG 这些很常见的工业图片格式,另外也多亏了最核心的代码是严格按照标准 C++去开发的,这让系统具备了极其优秀的跨平台编译能力,不论是想把它安装到 Windows 系统的工业电脑上,还是部署到基于 Linux 的边缘计算设备里,都能很灵活地去完成。

3.2.2 经济可行性

第一个方面是把部署和授权的成本给有效地控制住了,因为系统最核心的图像处理模块完全是靠自主研发做出来的,所以在工厂实际进行部署的时候,就不需要再给像 Halcon 或者 VisionPro 那些第三方的商业视觉软件去交一大笔昂贵的运行环境授权费了;另外,通过对算法进行优化,把系统运行时的资源消耗也给大幅度降下来了,这样哪怕是那些普通的常规工业电脑也能完全带得动它,根本用不着再去花大价钱买什么高端的计算硬件,进而就把企业最开始投进去的资金以及后期的日常运营成本都给大幅度削减了;

第二个方面体现在产业替代以及带来的市场价值上,就现在的行业大环境来看,半导体的制造还有检测设备都在加快实现国产化的步伐,在这样的大背景下,像这种既具备极高性价比,又完全拥有自主知识产权的缺陷检测软件,在市场上肯定会有着非常广泛且迫切的需求,可以说这个系统其实就是提供了一个既省钱又高效的视觉检测替代办法,如果有那些中小型芯片封装企业想要把自动化的质检水平给提上去,把这个系统用起来就能带来十分明显的经济效益。

3.2.3 操作可行性

在交互设计上做到了直观且合乎规范,系统的前端运用了图形化界面的设计,把整体布局顺应着主流工业软件的习惯,合理地分成了文件管理栏、图像展示主区域、参数配置侧边栏以及分析结果区;另外,把核心的检测流程都做成了直观可见的控制按钮,这样哪怕是没有专业开发经验的人员,经过简单的培训之后也能熟练地把系统操作起来,从而把工业现场需要人工介入的难度给有效地降下来了。

3.3 运行需求

1. 操作系统: Ubuntu 20.04 LTS 或 Windows 10/11 (64 位)
2. CPU: 多核处理器(≥ 4 核, 主频 $\geq 3.0\text{GHz}$)

- 3. 内存：4 GB 及以上
- 4. 依赖库：Qt 库：Qt6 Widgets
C++标准库：C++11 及以上

4 系统总体设计

4.1 系统整体架构设计

本芯片缺陷检测系统在架构设计上遵循“高内聚、低耦合”的软件工程原则，采用分层架构模式进行整体设计。系统自上而下主要分为表示层（UI 层）、业务逻辑层与算法数据层。如图 4.1 所示。

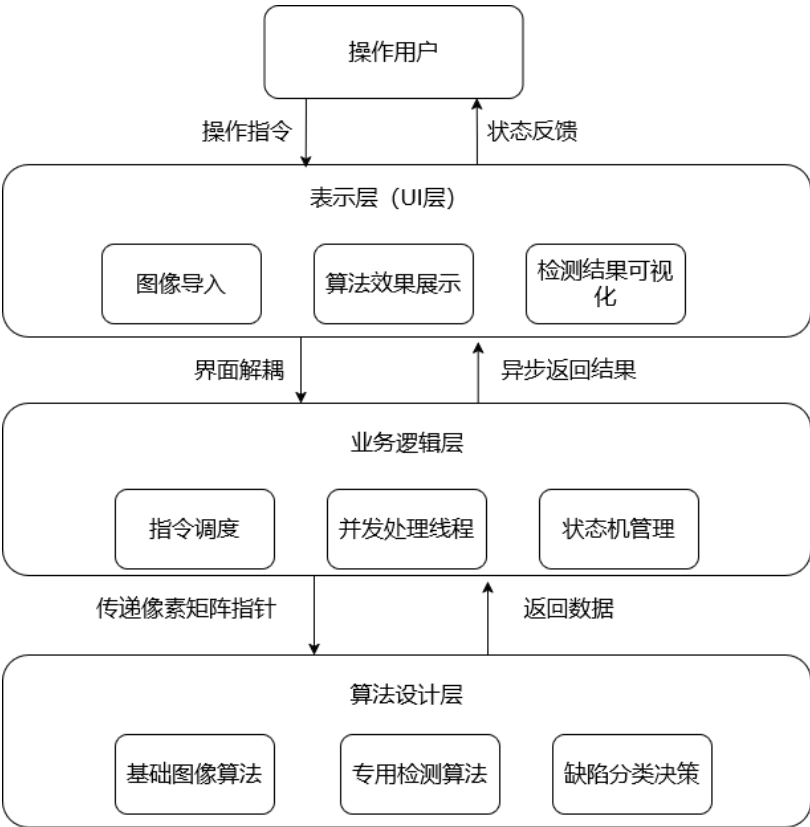


图 4.1 系统整体架构

表示层：主要负责前端的交互功能。该部分实现了原始图像在界面的展示、检测参数的实时输入，以及系统识别出缺陷位置后的可视化标定。

业务逻辑层：负责整体的任务调度与流程控制。考虑到图像矩阵运算比较耗时，本层利用多线程技术将计算任务单独剥离出去。这种方式避免了复杂算法执行时导致的用户界面卡顿，实现了 UI 渲染与后台处理的有效解耦。

算法数据层：主要包含视觉检测的核心代码逻辑。这里集中实现了空间滤波、边缘提取、形态学分析等基础图像处理算法，并且在编写时不依赖任何图形界面库。

采用上述典型的分层架构模式，明确了各个模块的具体调用关系。这不仅有效降低了系统代码的耦合度，也为后续核心算法模块的独立测试与跨平台移植提供了很大的便利。

4.2 系统功能模块设计

根据前期的需求分析与芯片缺陷检测的实际作业流程，本系统主要划分为五个核心功能模块：文件导入、预处理与模板匹配、连通域分析、缺陷分析以及结果输出。系统主要功能模块如图 4.2 所示。

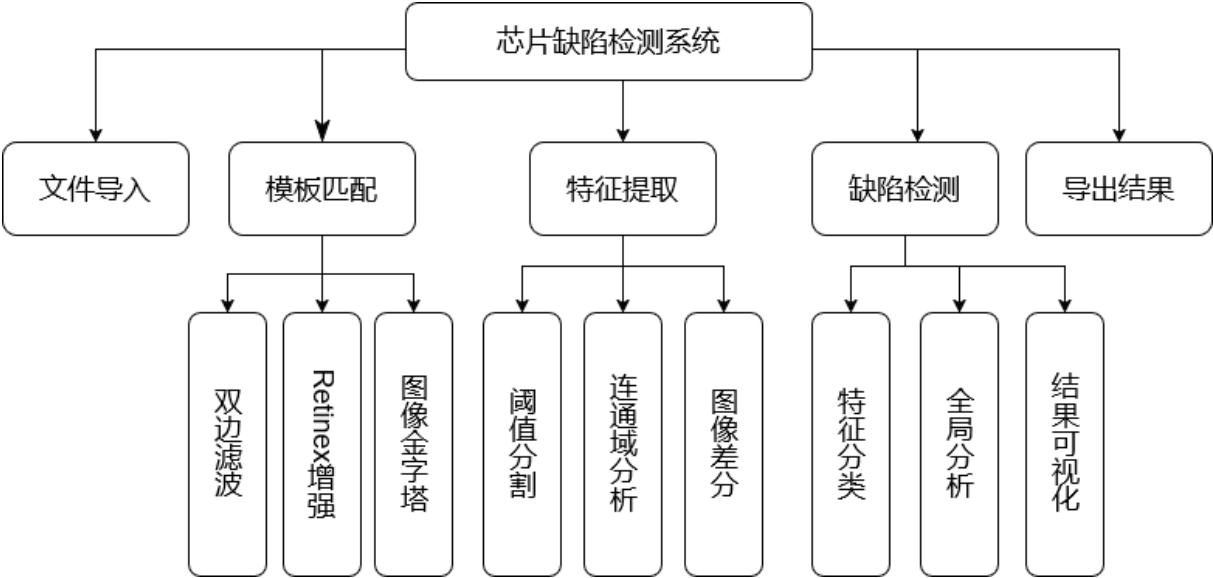


图 4.2 系统功能模块

文件导入与管理模块：系统能够读取和解析常见的工业图像格式。用户利用界面的文件选择功能，可以批量把待检测的芯片图片加载到主显示窗口中。为了方便对样本数据进行规范化管理，建议采用分层级的文件夹目录，将图像主要分为 defect（缺陷图片集）和 standard（标准模板）两类。

预处理与模板定位模块：本模块包含了多种基础的图像处理功能。如果遇到光照变化或噪点较多的情况，用户可以在界面上勾选使用高斯滤波、双边滤波、Retinex 光照补偿或者图像锐化等操作。同时，系统通过建立图像金字塔模型，实现了不同分辨率下的快速模板匹配与定位。

特征提取模块：主要负责从二值化到缺陷目标分离的完整流程。在利用阈值分割把背景剥离后，程序会调用连通域算法去分析剩余像素块的连通状态。这一步能有效过滤掉细微的背景噪点，把疑似缺陷的区域单独标记出来，并计算出它们的几何参数。

缺陷检测模块：这是软件进行判断的核心部分。系统会根据上一步提取到的连通域特征（例如面积、长宽比、紧凑度等指标），来识别该区域具体是划痕还是崩边，并据此统计产品的整体良率。检测步骤结束后，软件前端界面会立刻用标识框把缺陷位置圈出来，方便人工进行直观复查。

结果导出模块：主要用于将最终的检测结论、缺陷分类情况以及相关的统计数据，按照固定格式进行保存和导出。

4.3 系统核心类与运行机制设计

为确保操作员在产线环境下能够获得流畅的人机交互体验，同时保证底层视觉算法的高效执行，本系统对核心处理类和并发机制进行了专门设计。

4.3.1 用户交互与调度机制

主窗口控制类（MainWindow）。它主要是当作系统展示画面的核心地带，直接从 Qt 框架里的 QMainWindow 类继承过来；它发挥的作用涵盖了去搭建各类看得见的界面部件（像菜单栏、状态栏以及图像显示窗口），还要负责把用户交互产生的指令给顺利传递下去，并且把检测得出的结果给解析好再呈现出来；等接收到检测已经做完的信号之后，这个类就会去把那些加了具体说明标记的图片，还有到底合不合格（OK/NG）的判定状态，都非常稳妥地更新到前端页面上；

并发处理线程类（ProcessingThread）。系统特意引入了 QThread 机制，是为了去解决那些极其复杂的视觉计算步骤容易把主线程给完全卡住的麻烦；只要用户下达了开始检测的命令，主线程就会把图像矩阵的相关引用给转交到子线程的手里；接着这个类就会在系统后台独立地去调用检测算法，等所有步骤都运算完以后，再利用 Qt 特有的跨越线程发送信号的手段，把打包封装好的结果数据（DetectResult）给安全地传送回主窗口那里，通过这种做法就能极其有效地把软件界面突然卡死不动的现象给彻底规避掉了。

4.3.2 核心算法与数据处理机制

缺陷分析逻辑类（DefectAlgorithm）。这个类专门采用了一种没有状态设定的设计方式，主要是去充当整个图像处理运转流程的指挥调度员；一方面，它会把图像对齐、连通域提取、算一算形态学特征以及依靠专家决策树去分类这些本来各自独立的操作步骤，都给顺畅地串联到一起；另一方面，它能靠着一个统一的数据传输载体去跑通所有的流程，这样就把多线程运行环境底下常常容易产生的数据争抢冲突，还有那些完全没必要的内存深层次拷贝所带来的性能消耗，都给极其有效地减少了；

底层的图像处理类（PixelProcessor）。它主要是把那些专门对像素矩阵去做空间或者频域操作的基础计算接口都给整体打包封装了起来；这里面具体涵盖了高斯平滑滤波、Retinex 光照均值化、非常快速的 Sobel 边缘提取，以及自适应的局部阈值分割这些极为实用的方法；另外在具体的代码落地细节上，这个模块还巧妙地把内存指针偏移与循环展开这类的底层代码优化策略给结合了进去，进而把那些基础计算方法实际跑起来的执行效率给大幅度地提升了。

4.4 系统界面结构设计

为了提升缺陷检测系统的易用性与操作效率，界面设计遵循了可见性、操作一致性及状态及时反馈的原则。前端页面整体布局规划如图 4.3 所示。

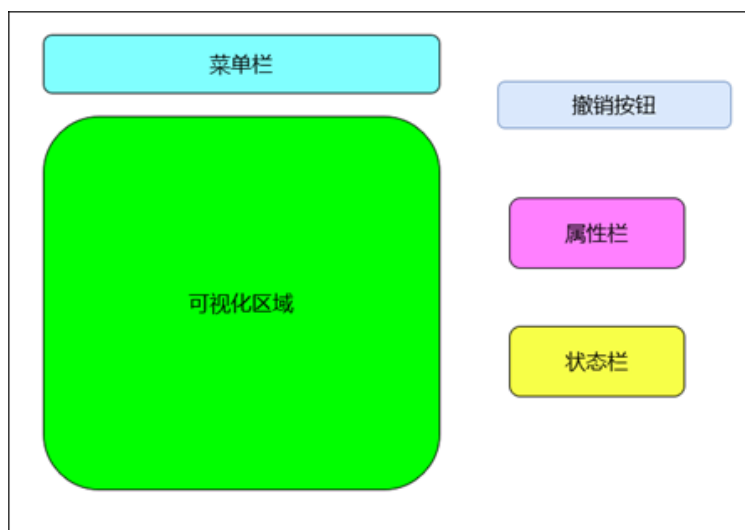


图 4.3 界面设计

菜单栏与工具栏：采用分组设计，包含文件系统操作（导入/导出）、预处理工具、模板匹配、连通域提取及缺陷分析等核心功能项，功能层级清晰，便于用户快速调用。

核心可视化视窗：占据界面主体的图像渲染区。除了实时展示当前处理状态外，还支持鼠标滚轮缩放、中键视角拖动及快捷键翻页等高级交互功能，方便用户对微小缺陷进行局部细节审查。

操作回滚机制（撤销功能）：系统内部维护了历史状态栈，当用户进行误操作时，可通过撤销按钮快速回滚至上一级有效状态。

属性检视面板：动态显示当前工作区图像的元数据，包括分辨率尺寸、色彩通道数、位深度等。

状态监控栏：位于界面右部，实时滚动显示系统当前执行的任务流转状态和最终的 OK/NG 判定结果。

配合上述功能，系统推荐在本地部署如下的工程文件目录结构（如图 4.4），以保障数据读写路径的一致性。

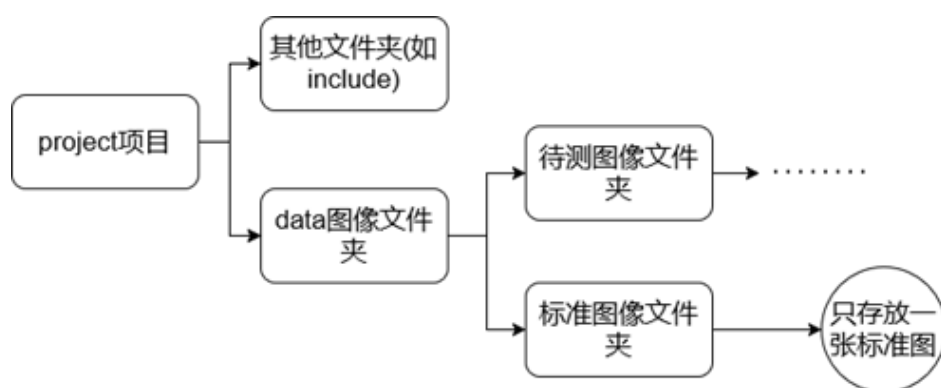


图 4.4 文件目录结构

5 系统详细设计及编码

本章主要讲述详细设计思路以及核心代码展示，关于各个功能模块的具体实现。包括：文件导入、模版匹配、前景提取和结果导出。

5.1 文件导入

5.1.1 功能概述

图像导入模块是芯片二维视觉检测系统的数据入口，负责将外部工业相机采集或本地存储的图像数据高效解析，并映射为系统底层算法所需的自定义数据结构。本模块的核心功能：支持工业视觉领域主流的 BMP、PNG、JPG 等标准二维图像格式的读取与加载。自动剥离图像文件头，将纯像素数据提取并重构为底层检测算法依赖的自定义一维数组模拟二维矩阵结构。实时渲染导入的芯片图像，支持视图容器内的平移、缩放操作，并提供文件列表的快速切换与管理功能。

5.1.2 图像导入与渲染流程

系统运行并完成初始化之后，用户可以利用主界面上的菜单栏或快捷工具按钮来导入待检测的图片。主要的操作与处理过程如下：

当用户点击“文件”菜单中的“打开图像”选项时，程序会弹出 Qt 自带的 QFileDialog 文件选择对话框。

在用户选定目标文件并确认后，系统便开始进行加载和界面显示。前端主要依靠 Qt 的 QImage 或 QPixmap 类，将图片迅速展示在主操作区域（Graphics View）内。与此同时，后台程序会读取这张图像的宽度、高度以及像素的灰度值，并根据这些尺寸信息在内存中申请合适的空间，将其转换成后续算法能够直接调用的矩阵形式。具体的导入流程如图 5.1 所示。

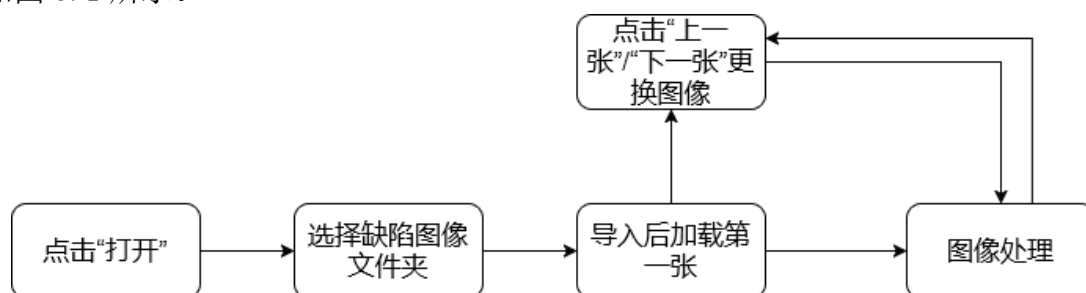


图 5.1 文件导入

图像在界面加载成功后，用户能通过鼠标方便地查看画面细节。例如，滑动滚轮可对局部区域进行缩放，以便观察微小缺陷；若图片尺寸超出当前窗口，按住左键拖动即

可平移图像。同时，软件界面的状态栏与侧边面板会自动显示该图片的基本信息，主要包括当前的色彩空间和长宽分辨率。这些实时更新的参数，为后续的人工作业与观察提供了直观的数据参考。

5.1.3 界面展示与检测流程

首先，点击“打开”按钮后会自动进入到项目路径。如图 5.2 所示。

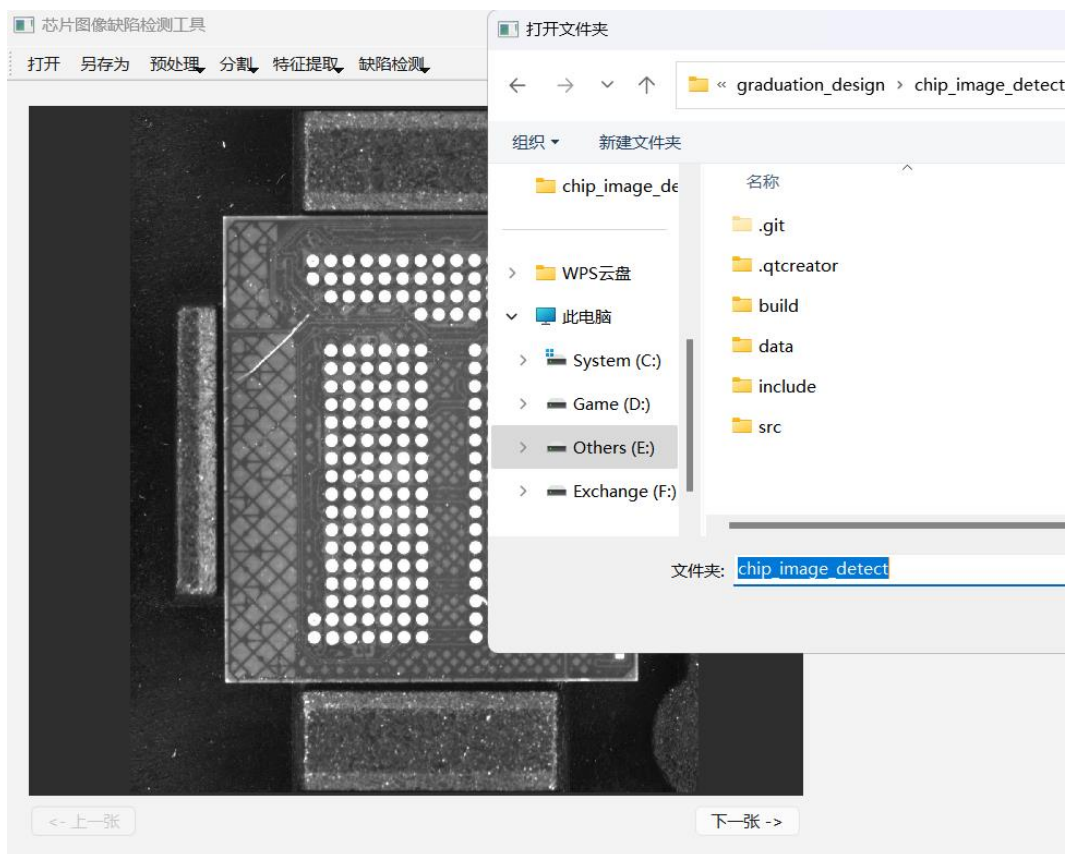


图 5.2 导入图像

支持图像拖动，缩放等功能，也可以点击右下角“下一张”或左下角“上一张”快速查看图像。在此之后，在“缺陷检测”按钮处可以依次点击“模板匹配”、“阈值分割”、“连通域分析”、“图像差分”、再次连通域分析和“缺陷分析”。

检测算法流程图如图 5.3 所示。

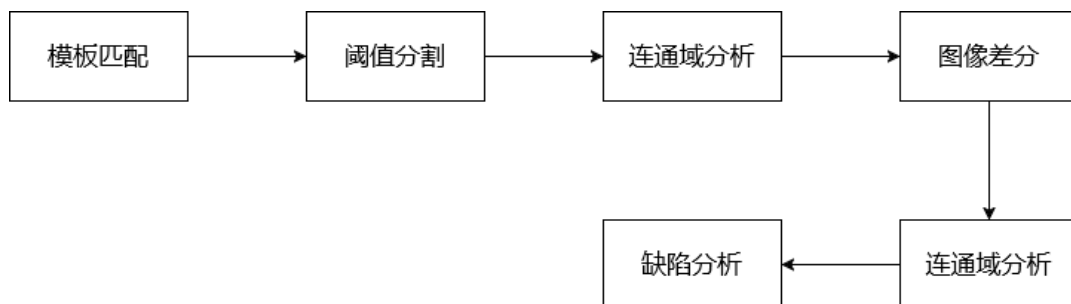


图 5.3 检测流程

5.2 模板匹配

本小节实现了待测图和标准图的匹配，涉及滤波、图像增强、图像金字塔步骤，目的是初步实现芯片定位和预处理，减少光照、噪声等因素的干扰。

5.2.1 双边滤波

常见的高斯滤波等线性平滑算法，主要根据像素间的空间距离来分配权重。这种方法在去除噪声的同时，往往也会把图像边缘的细节给模糊掉。考虑到芯片表面的细小划痕和崩边在图像中通常表现为灰度值剧烈变化的边缘特征，如果直接使用线性平滑处理，很容易把这些真正的缺陷当成噪点一起抹平，从而降低系统的最终检测精度。

为了解决这个问题，本系统在预处理阶段选择了双边滤波（Bilateral Filtering）算法进行降噪。这种算法不仅考虑了像素的空间距离，还把像素之间的灰度差异也加到了权重的计算当中。当滤波窗口在灰度比较均匀的芯片背景上移动时，算法能起到很好的平滑去噪作用；而当窗口经过缺陷边缘时，由于相邻像素的灰度相差很大，算法会自动降低周边像素的权重。

这种结合了空间和灰度双重信息的处理方式，有效防止了边缘两侧像素的相互模糊。从实际的运行效果来看，双边滤波不仅较好地抑制了拍摄时产生的背景杂波，还清晰地保留了微小缺陷的轮廓特征。通过这一步良好的预处理，去除了不必要的数据干扰，为后面进行图像分割和缺陷坐标提取打下了坚实的基础。以下是核心代码。

```

Algorithm: twoWayFilter
Input: 输入灰度图像  $I$ , 窗口半径  $R$ , 空间域标准差  $\sigma_s$ , 值域标准差  $\sigma_r$ 
Output: 降噪后的边缘保留图像  $Result$ 
// 阶段 1: 预计算 Domain (空间域) 权重矩阵
1: 初始化空间权重矩阵  $G_{space}$ , 大小为  $(2R+1) \times (2R+1)$ 
2: for  $dy = -R$  to  $R$  do
3:   for  $dx = -R$  to  $R$  do
4:     计算几何距离  $d = \sqrt{dx^2 + dy^2}$ 
5:      $G_{space}[dx, dy] = \exp(-d^2 / (2 * \sigma_s^2))$  // 高斯函数
6:   end for
7: end for
// 阶段 2: 空间与值域联合降噪遍历
8: 获取图像宽度  $W$  和高度  $H$ , 初始化结果图像  $Result$ 
9: for  $y = 1$  to  $H$  do
10:  for  $x = 1$  to  $W$  do
11:     $I_{center} = I[x, y]$  // 当前中心像素灰度值
12:    初始化累加权重  $Sum\_W = 0$ , 累加像素值  $Sum\_Pix = 0$ 
13:
14:    // 遍历  $(2R+1) \times (2R+1)$  局部邻域  $\Omega$ 

```

```

15:         for dy_n = -R to R do
16:             for dx_n = -R to R do
17:                 // 边界安全检查，跳过越界像素
18:                 如果 (x+dx_n, y+dy_n) 越界, 则 continue
19:
20:                 I_neighbor = I[x+dx_n, y+dy_n] // 邻域像素灰度值
21:
22:                 // --- 联合滤波核心 ---
23:                 // a. 获取空间权重 (Domain Weight): 只看距离, 不看颜色
24:                 W_space = G_space[dx_n, dy_n]
25:
26:                 // b. 计算值域权重 (Range Weight): 只看颜色差, 不看距离
27:                 // 对颜色差异大的像素 (边缘) 给予低权重, 从而保留边缘
28:                 Diff = I_neighbor - I_center
29:                 W_range = exp(-Diff^2 / (2 *  $\sigma_r^2$ ))
30:
31:                 // c. 计算总权重并累加
32:                 W_total = W_space  $\times$  W_range
33:                 Sum_W = Sum_W + W_total
34:                 Sum_Pix = Sum_Pix + (W_total  $\times$  I_neighbor)
35:             end for
36:         end for
37:
38:         // 阶段 3: 归一化输出
39:         Result[x, y] = Sum_Pix / Sum_W
40:     end for
41: end for
42: return Result

```

主要参数说明:

σ_s	平滑的空间范围，类似于高斯滤波。越大，参与平滑的邻域越广，图像整体越模糊。
σ_r	滤波器对灰度差异的“宽容度”。滤波器对边缘越敏感，边缘保留效果越好，但降噪能力变弱；越大，双边滤波将退化为普通的高斯滤波。

5.2.2 Retinex 增强

根据颜色恒常性理论，一幅图像可以看作是由环境光照和物体表面反射两个独立分量组成的。针对由于车间光照不均导致图片暗部细节不清晰的问题，本系统使用了单尺度 Retinex (SSR) 算法来增强图像亮度并恢复细节。

在编写算法代码时，程序首先遍历一次灰度矩阵，计算并保存积分图。在计算局部均值时，利用这张积分图，可以将任意大小邻域内的像素求和操作转换为查表和简单的加减运算。这种以空间换时间的做法，让局部均值的计算时间复杂度降到了 $O(1)$ 。计算得出的局部均值，就可以直接作为该像素点光照分量 L 的估计值。

程序接着将原始灰度值 S 和算出的光照分量 L 放入对数空间中相减，也就是计算 $\log(S) - \log(L)$ 。经过这一步计算，就能去除外部光照不均带来的影响，提取出反映芯片表面真实纹理和缺陷的反射率分量 R 。

最后一步，程序会找出矩阵中的最大和最小值，通过线性拉伸公式将数据重新映射回 0 到 255 的常规灰度区间。测试结果表明，使用积分图加速的 SSR 算法，既能把处于暗处的细小划痕显示清楚，又保证了较快的图像处理速度。核心伪代码如下。

Algorithm: Retinex

Input: 原始图像 I

Output: 增强后的图像 $Result$

// 阶段 1: 初始化与构建积分图

1: 将图像 I 转换为灰度图像 $Gray$ ，获取图像宽度 W 和高度 H

2: 初始化积分图矩阵 $IntImg$ ，大小为 $(W+1) \times (H+1)$ ，全置为 0

3: for $y = 1$ to H do

4: for $x = 1$ to W do

5: // 计算当前点及其左上角所有像素的累加和

6: $IntImg[x, y] = Gray[x, y] + IntImg[x-1, y] + IntImg[x, y-1] - IntImg[x-1, y-1]$

7: end for

8: end for

// 阶段 2: 光照估计与 Retinex 核心计算

9: 确定感受野半径 $radius = \min(W, H) / 5$

10: 初始化对数域反射分量矩阵 R ，大小为 $W \times H$

11: 初始化极值变量 $max_R = -\infty$, $min_R = +\infty$

12:

13: for $y = 1$ to H do

14: for $x = 1$ to W do

15: 确定以 (x, y) 为中心， $radius$ 为半径的有效局部窗口边界 $(x1, x2, y1, y2)$

16: 窗口面积 $Area = (x2 - x1) \times (y2 - y1)$

17:

18: // 利用积分图在 $O(1)$ 时间内查表求得窗口像素总和

19: $Sum = IntImg[x2, y2] - IntImg[x1, y2] - IntImg[x2, y1] + IntImg[x1, y1]$

20:

21: // 计算局部均值作为环境光照估计 L

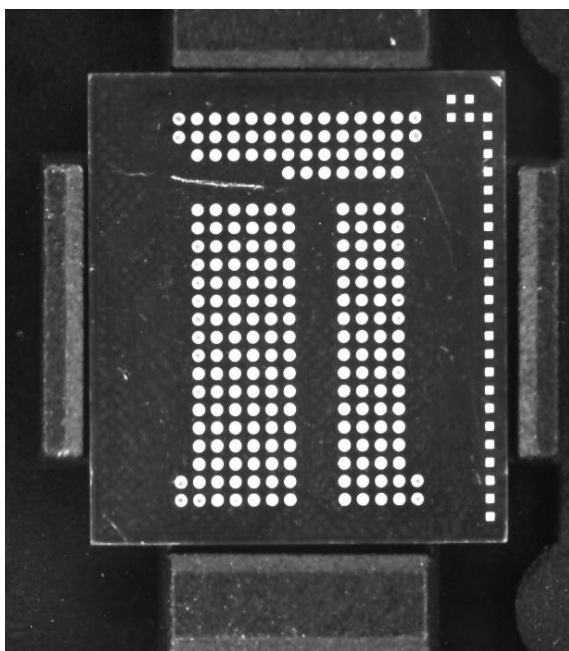
22: $L = Sum / Area$

```

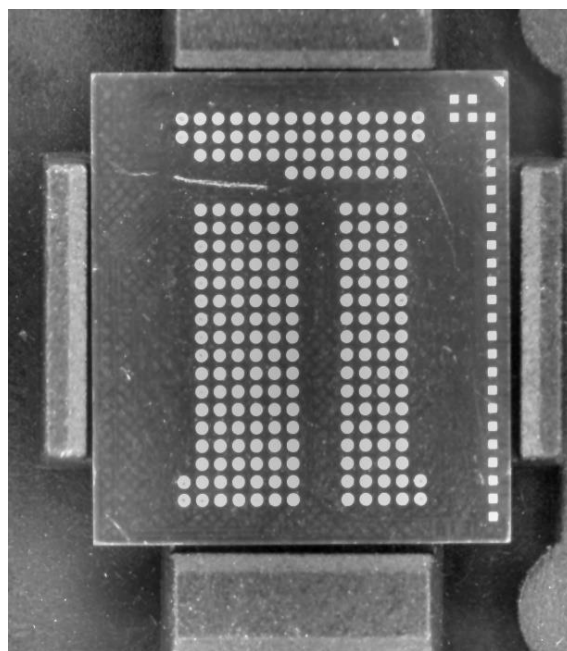
23:         S = Gray[x, y]
24:
25:         // 根据 Retinex 理论计算反射分量 (加 1 防止 log(0))
26:         R[x, y] = log(S + 1) - log(L + 1)
27:
28:         // 记录 R 矩阵的全局最大值和最小值
29:         如果 R[x, y] > max_R 则 max_R = R[x, y]
30:         如果 R[x, y] < min_R 则 min_R = R[x, y]
31:     end for
32: end for
// 阶段 3: 线性拉伸映射
33: // 防止出现纯色图像导致除以 0 的异常
34: 如果 (max_R - min_R) 趋近于 0, 则令区间差值为 1
35:
36: for y = 1 to H do
37:     for x = 1 to W do
38:         // 将浮点数值线性映射回 8 位灰度空间
39:         Result[x, y] = (R[x, y] - min_R) / (max_R - min_R) × 255
40:         将 Result[x, y] 截断限制在 [0, 255] 范围内
41:     end for
42: end for
43: return Result

```

算法效果展示:



(a)处理前



(b)处理后

图 5.5 Retinex 增强

5.2.3 图像金字塔

在处理高分辨率图片时，如果直接使用全局滑动窗口进行模板匹配，会导致计算开销过大。为了解决这个问题，本模块采用了多尺度图像金字塔结构，通过一种“由粗到精”的搜索方式来节省计算时间。

在代码运行的初始阶段，程序首先利用下采样方法，生成了包含三层分辨率的图像金字塔：即原始大小的原图（L0）、长宽缩小一半的图像（L1）以及长宽缩小至四分之一的图像（L2）。实际的匹配过程直接从最顶层的 L2 层开始。由于 L2 层的图像总面积只有原来的十六分之一，所以系统能够用很少的计算量，迅速找出目标区域的大致位置。

得到粗略位置后，程序就开始逐层向下进行坐标映射和位置微调。具体的做法是，把上一层找出的坐标位置乘以相应的缩放比例，换算到下一层图像里。然后，以这个映射过来的新坐标为中心，在一个较小的局部范围内再次进行模板匹配。这个向下投影并局部搜索的过程会一直重复，直到回到最底层的 L0 原始图像，从而得到目标像素级别的精确坐标。

另外，在根据这个精确坐标去裁剪和提取目标区域之前，代码中加入了针对图像宽高尺寸的边界判断逻辑。这一步主要是为了防止由于坐标超出图像范围而导致内存越界报错。总体来看，这种基于金字塔降维的搜索方法，既保证了目标定位的准确性，又有效缩短了模板匹配的时间，满足了检测系统对处理速度的实际要求。算法原理如图 5.6

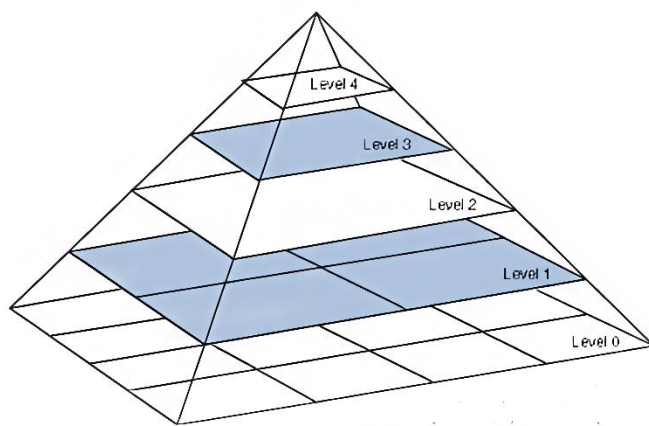


图 5.6 图像金字塔

算法效果如下。

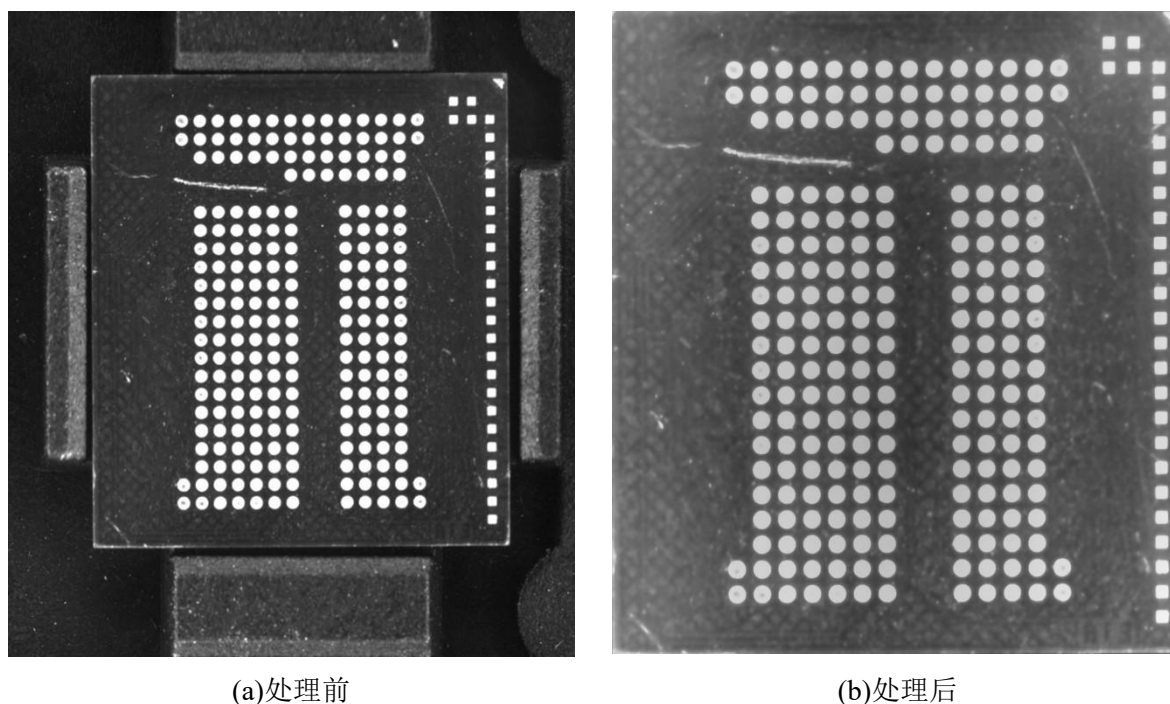


图 5.7 图像匹配

5.3 特征提取

该模块整体执行逻辑由阈值分割、连通域分析、以及图像差分承前启后的子步骤构成。

5.3.1 阈值分割

在图像的自适应二值化处理阶段，本模块使用大津法（Otsu）来自动计算最佳的分割阈值。程序首先会对图像矩阵进行一次完整的遍历，统计 0 到 255 每个灰度级出现的次数，并计算出相应的概率分布。通过这种生成直方图的预处理方式，后续寻找最佳阈值的过程就不需要反复读取二维图像，而是直接在一个长度为 256 的一维数组里进行计算。

在寻找最佳阈值时，程序会让阈值变量 k 在 0 到 255 之间逐个递增。为了减少重复计算，代码在循环中利用了前一次的缓存结果，通过递推的方式快速算出当前 k 值下的前景概率、背景概率以及均值。随着 k 值的增加，程序会根据公式同步计算出当前的类间方差。在统计学上，类间方差越大，说明目标和背景区分得越明显。

遍历完这 256 个灰度级后，程序会找出使得类间方差最大的那个数值，并把它确定为当前图像的最优阈值 T 。最后，利用这个算出来的阈值对原图像的每个像素进行判断，从而得到一张目标特征清晰的二值化图像。这种完全根据图像本身灰度分布来自动寻找阈值的方法，在实际测试中较好地克服了车间光照变化带来的影响。

效果如图 5.8

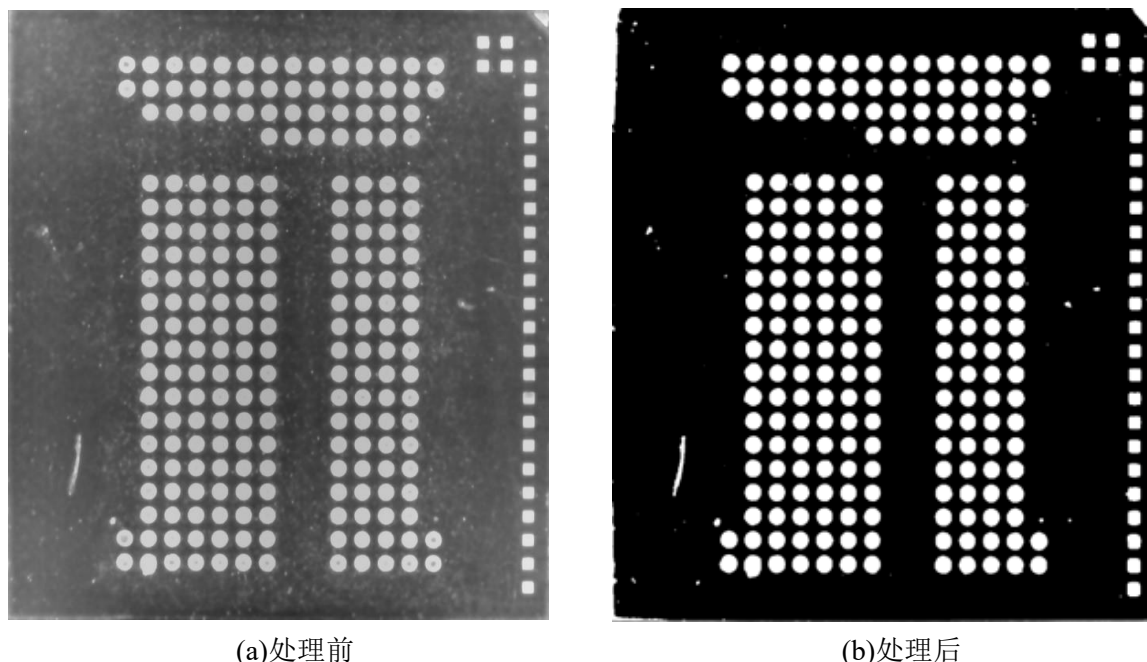


图 5.8 阈值分割

5.3.3 连通域分析

提取拓扑特征前，必须先内存中构建出完整的连通域标记矩阵。具体到代码编写上，对二值图像的第一次扫描是严格按行推进的。当指针查找到前景像素时，程序会去核对它正上方和左侧像素的标记状态。如果这两个参考点都是空白的，就给当前坐标分配一个新的递增标号；如果仅有一边带标签，就直接顺延继承；但如果刚好碰到了“U”型分支的交汇口，邻域内往往会产生两个不同的临时编号。针对这种冲突，底层代码会触发并查集（Union-Find）机制，在专门的等价关系数组里，把这两个标签强制挂靠到同一个根节点上。

由于这种单向扫描必然会导致同一个物理缺陷挂着多个冗余标签，在启动二次遍历之前，程序专门执行了一次路径压缩。这一步操作主要是把等价关系树进行展平，剔除合并时产生的断层跳号，重新整理出一套从 1 开始连续递增的映射关系表。等到第二轮询图像像素时，代码直接去查这个映射表，把矩阵里遗留的临时编号全部覆写替换掉，从而在内存层面彻底将各个独立的连通区块给切分干净。

把独立区块分离出来之后，下一步的重心就是怎么从一堆候选图形里，把真正的管脚缺损和表面划痕挑出来，并且屏蔽掉环境带来的伪影干扰。这里的处理主要依靠形态学参数来约束。程序对标记好的矩阵进行了一次全局遍历，中间利用哈希表（Hash Map）来快速累加各个编号包含的像素点数量，以此算出缺陷面积，并同步更新每一个独立区块的最小外接矩形（Bounding Box）坐标。

拿到这些几何特征参数后，后端的筛选逻辑就开始结合工业现场的实际公差来跑了：程序会直接把面积太小的零散噪点和面积过大的异常背景给排除掉；紧接着，考虑到前

面图像配准阶段可能会残留下一些狭长的伪影边缘，代码里又追加了严格的长宽比阈值。只有同时通过了这两道特征筛查的缺陷编号，才会被保留在合法名单里。最后，程序依照这份“白名单”重写输出矩阵，把无效的像素点强行清零。这种依靠几何特征提前拦截的判定策略，能稳稳地输出一张只保留高纯度缺陷的二值图，给最终的系统判定提供干净的数据源。

以下是算法伪代码：

```

Algorithm: featureExtraction
Input: 连通域标签矩阵 L (宽 W, 高 H)
Output: 纯净的目标缺陷二值图 Result
//全局特征参数提取
1: 初始化特征集合 Stats, 用于记录每个连通域的 面积(Area) 与 外接矩形边界 (minX, maxX, minY, maxY)
2: for y = 0 to H - 1 do
3:     for x = 0 to W - 1 do
4:         ID = L[x, y]
5:         如果 ID == 0 (背景像素) 则继续下一次循环
6:
7:         // 累加面积并更新当前 ID 连通域的极值坐标
8:         Stats[ID].Area = Stats[ID].Area + 1
9:         Stats[ID].minX = min(Stats[ID].minX, x), Stats[ID].maxX =
max(Stats[ID].maxX, x)
10:        Stats[ID].minY = min(Stats[ID].minY, y), Stats[ID].maxY =
max(Stats[ID].maxY, y)
11:    end for
12: end for
// 形态学约束筛选
13: 设定面积约束阈值 T_min = 50, T_max = 6000
14: 设定长宽比约束阈值 T_ratio = 8.0
15: 初始化有效标签集合 ValidSet
16:
17: for 遍历 Stats 中的每一个连通域 C do
18:     // 计算外接矩形的长宽比 (恒定使用长边除以短边)
19:     宽度 W_c = C.maxX - C.minX + 1
20:     高度 H_c = C.maxY - C.minY + 1
21:     长宽比 Ratio = max(W_c, H_c) / min(W_c, H_c)
22:
23:     // 多维形态学规则判定
24:     如果 (C.Area ∈ [T_min, T_max]) 且 (Ratio ≤ T_ratio) 则:
25:         将 C.ID 加入 ValidSet (标记为真实缺陷)
26: end for
// 目标图像重建

```

```
27: 初始化输出二值图 Result, 尺寸 W × H, 全置为 0 (纯黑)
28: for y = 0 to H - 1 do
29:     for x = 0 to W - 1 do
30:         ID = L[x, y]
31:         如果 ID 存在于 ValidSet 中:
32:             Result[x, y] = 255 (保留为纯白前景)
33:         end for
34:     end for
35: return Result
```

核心参数说明:

T_{min} 和 T_{max} (面积阈值)	下限用于过滤微小孤立白点; 上限用于剔除大块伪影
T_{ratio} (长宽比阈值)	由于芯片边缘的微小错位, 极易在差分阶段沿引脚或边缘产生细长的伪缺陷。限制长宽比阈值以排除伪缺陷

算法效果展示:

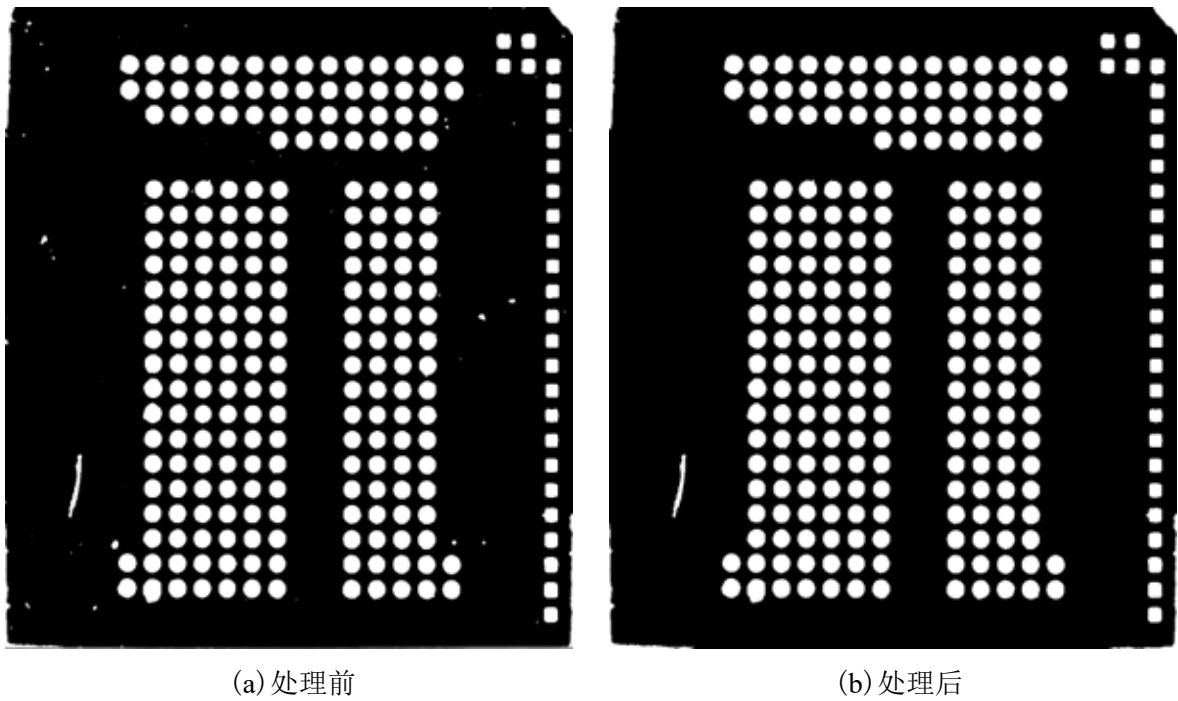


图 5.9 连通域分析

5.3.3 图像差分

在提取异常特征时, 往往需要依赖图像差分技术。但传统的绝对值差分对图像配准的精度要求过于苛刻, 轻微的机械振动或亚像素级的对齐误差, 都会导致芯片的正常边缘暴露出大量伪缺陷轮廓。为了解决这一痛点, 提高方案在实际工程中的鲁棒性, 我们

采用了一种基于十字形局部均值的空间容错差分机制。

具体处理时，算法不再直接读取单个像素的灰度去硬作差。程序在遍历有效像素区时，会提取当前像素及其上下左右组成的 5 邻域（十字窗口），分别计算测试图和模板图在该局部的灰度均值。这种局部平滑的做法，相当于在作差前垫了一个微型低通滤波器，能有效“柔化”并抵消掉由于图像微小错位带来的边缘抖动。

求出两者的局部均值并相减后，再通过截断函数将结果强制约束在 0~255 的常规范围内。至于差分图里残存的零星微小噪点，最后只需套用一次双边滤波进行清理即可。效果展示：

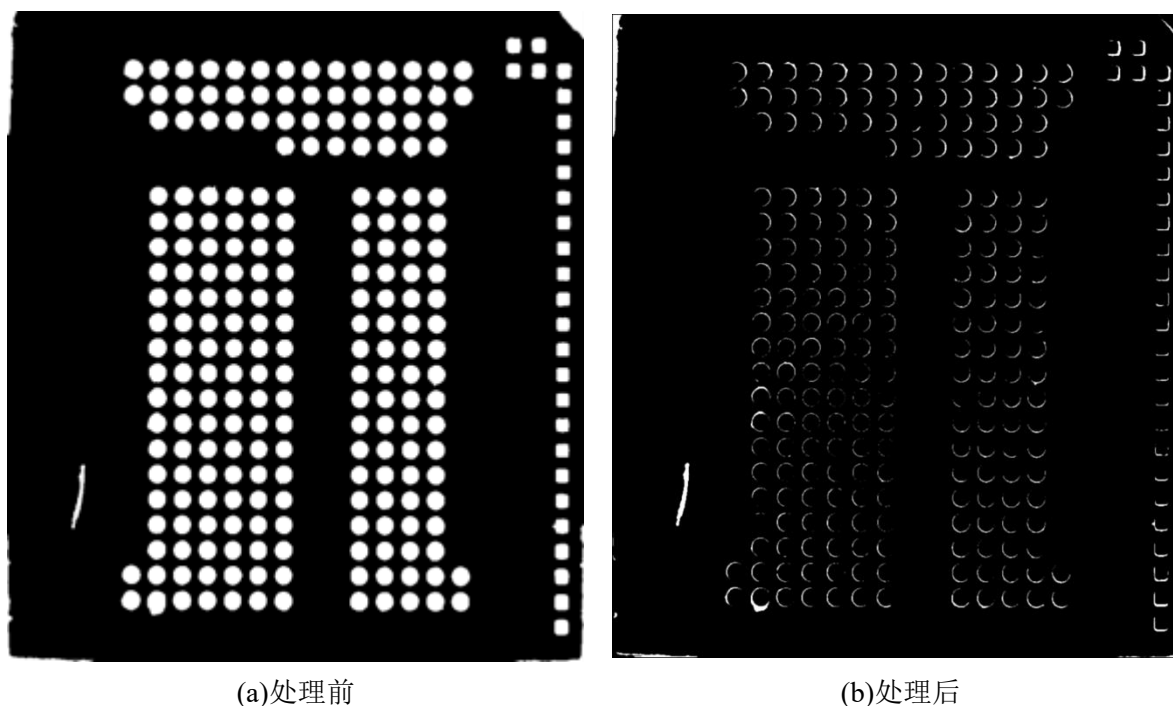


图 5.10 图像差分

5.4 缺陷检测

5.4.1 特征分类

把缺陷分离出来后，下一步就是给它们“定性”分类。在这个环节，我们没有采用复杂的模型，而是基于物理常识搭建了一套轻量级的规则判定逻辑（相当于一个简单的专家决策树）。

我们在判定时主要看两个维度。第一个是边缘位置。因为芯片在晶圆切割或机械贴片时，边缘最容易受应力冲击而碎裂，所以我们在图像四周划定了一个宽度为 10 像素的“边缘警戒区”。只要缺陷的边界框碰到了这个区域，哪怕只沾到一点，也会直接被赋予最高危险等级，判定为“崩边”。

如果异常点没在边缘，而是落在芯片内部的安全区域，我们就通过长宽比（Aspect

Ratio) 这个形态指标来做进一步细分。当目标的长宽比超过设定的几何阈值(本项目设为 3.0) 时, 说明这个缺陷明显偏细长, 直接归类为表面“划痕”。反之, 如果长宽比相对均衡, 看着像个普通的块状斑点, 就统一归类为生产过程中沾上的异物或表面污损。

这套基于硬阈值的判定逻辑虽然简单, 但计算开销极低。它能在满足工业级实时性要求的前提下, 把这几种常见缺陷又快又准地筛分出来。分类流程图如下:

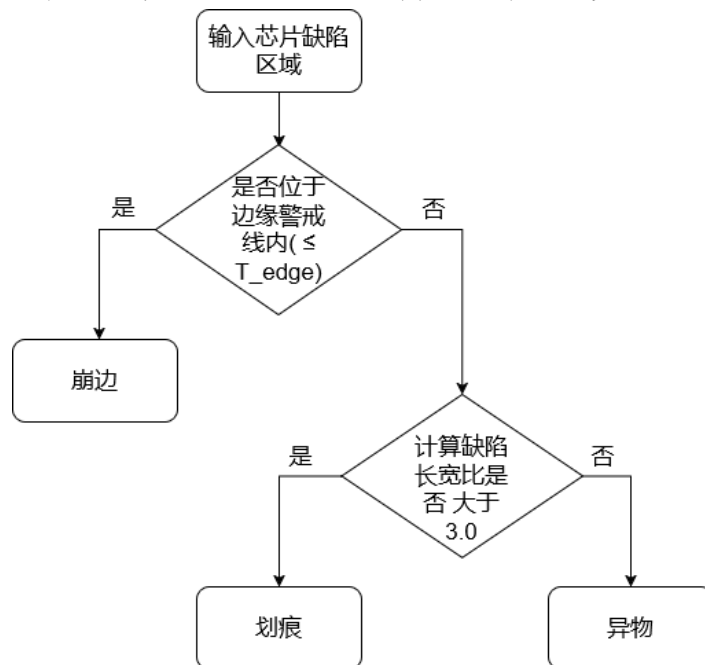


图 5.11 特征分类流程

5.4.2 全局分析

要想把整片芯片的生产质量给全面综合地评估清楚, 系统就需要从全局的角度去考虑问题, 因为光靠数一数缺陷到底有几个, 或者单看面积有没有超过设定的那条界线, 往往没办法真正反映出工业产品到底还能不能用; 第一点, 为了搞定这个事情, 本项目在全局判定模块当中, 专门去搭建了一套模仿人工质检专家判断习惯的双轨决策模型, 也就是“定性一票否决加上定量面积累加”机制; 一方面, 在检查遍历的时候, 一旦系统察觉到了划痕或者是崩边这类破坏性的毛病, 考虑到这些物理上的损伤极其容易引发芯片内部的电路彻底断掉, 或者引起封装应力全部集中到一起, 系统就会把面积大小这个因素给完全忽略掉, 直接给出一个不合格(NG)的判决结果; 另一方面, 等把那些致命的损伤给排除掉以后, 系统就会立刻转到定量评估的运作机制里去, 算法会把累加起来的受损总面积, 去跟提前设置好的容差标准(本项目给设定到了 3000 像素)拿来仔细比对一下; 要是算出来超标了, 就会被判定成不良品(NG), 只有当芯片身上既找不出那些致命的结构损伤, 同时表面弄脏坏掉的总面积也被牢牢地控制在安全的界线范围之内时, 系统才会去下达那个最终认定为合格(OK)的指令; 总的来看, 把这套双轨决策的体系给用上, 既照顾到了质检环节所必需的严苛程度, 又保留了工程良品率该有的

宽容空间，进而把一刀切算法极易造成的过度误杀或者是漏查漏检的问题给有效地规避掉了。

5.4.3 可视化结果输出

到了最后的可视化展示阶段，我们需要把找出的缺陷直观地画在图上。因为前面的差分和特征提取都在单通道灰度图上跑的，为了能画彩色标注，程序会先把原图转换成三通道的 RGB 格式，并顺手开启画笔的抗锯齿功能，防止画出来的框有毛边。

接下来就是把确认好的缺陷挨个“圈”出来。为了让质检人员一眼就能看清状况，我们按严重程度做了一套直观的颜色分级：像“崩边”这种会破坏芯片结构的致命缺陷，直接用醒目的红色粗线框标出；有断路风险的“划痕”，用黄色框提示；而像表面异物这类的普通瑕疵，用常规的蓝色框圈定就行。

在画边界框的同时，程序会根据坐标偏移，顺便在框的左上角打上对应的文字标签。这样处理完后，质检员拿到图，哪里有缺陷、是什么类型的缺陷，立刻就能一目了然。

以下是效果展示：

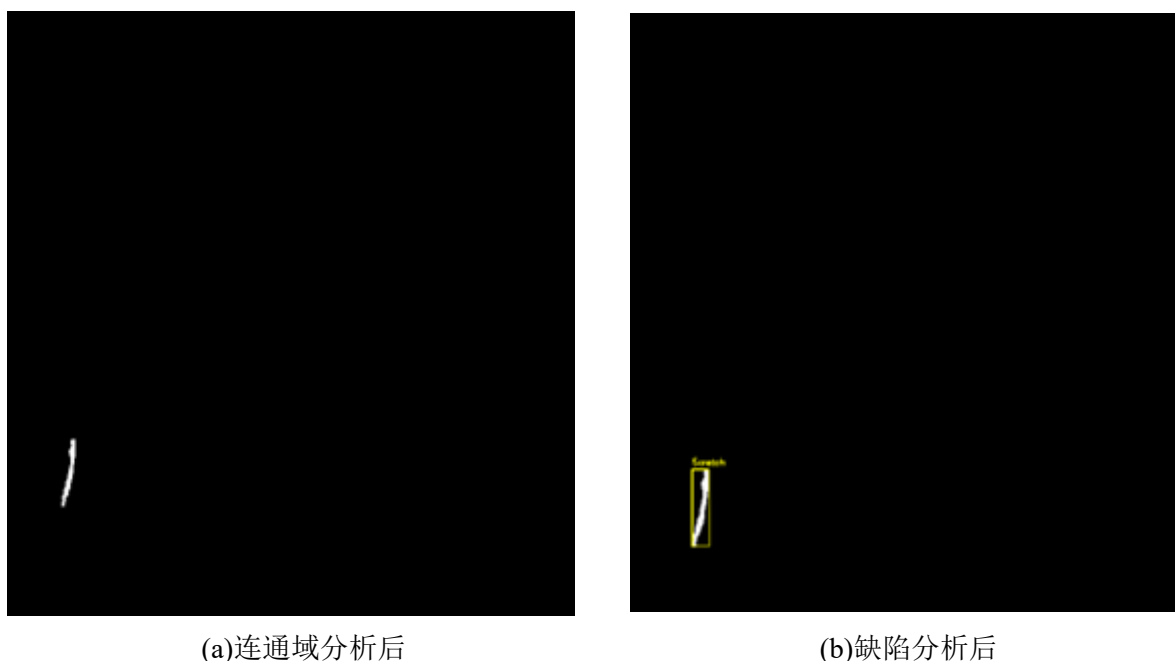


图 5.12 缺陷结果

5.5 导出结果和系统测试

在满足 3.3 运行需求的情况下，进行了系统测试。经过了文件导入、模板匹配、特征提取和缺陷检测环节后，系统成功地运行并且得到了正确的检测结果。在完成这一系列检测流程之后，可以将检测到的缺陷图像导出，选择“另存为”即可。也可以把检测得到的缺陷数据一并导出。

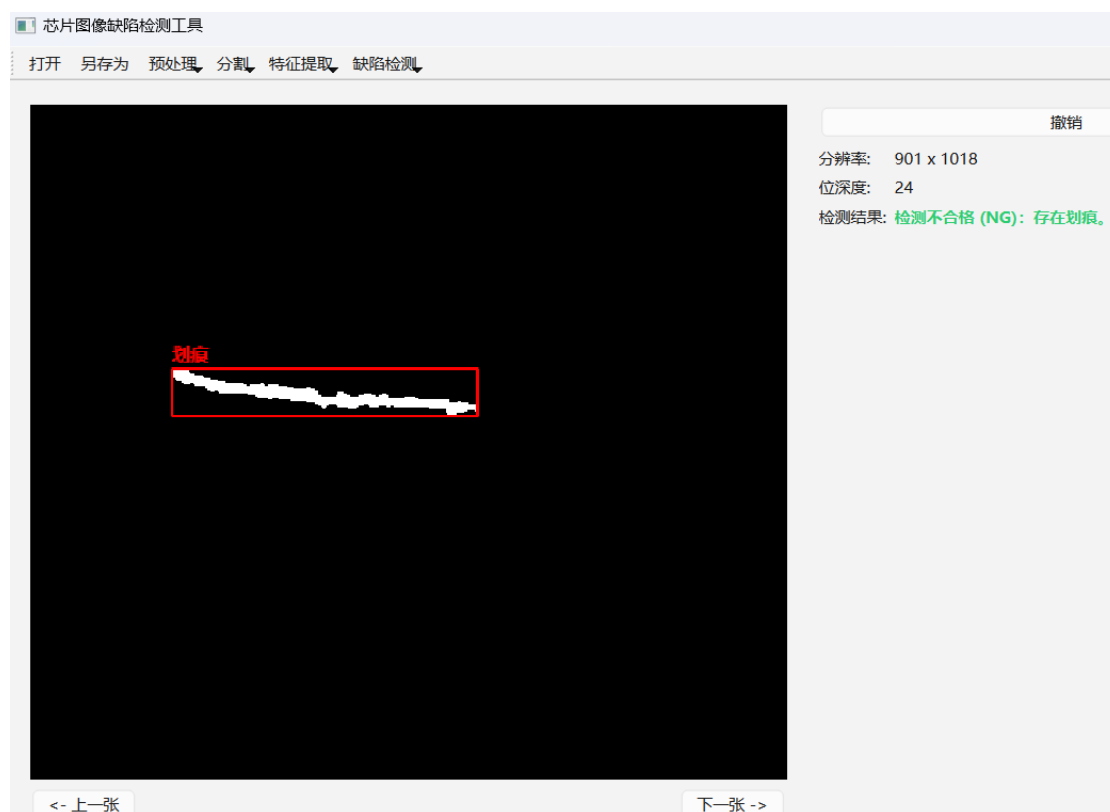


图 5.13 界面结果

如图 5.13 所示，最后得到的缺陷是划痕，已在结果图中用红色矩形框出并且在左上角标注红色字体。与此同时，右侧状态栏也给出了最终的检测结果——检测不合格(NG)：存在划痕。后续可以选择点击“另存为”将结果图像和检测数据保存在指定文件夹。

经过对本系统各模块的功能测试与性能测试，结果表明，系统运行稳定，各项功能均达到了预期设计要求，界面对用户友好，不存在严重的逻辑漏洞，具备一定的实用价值。

6 总结与展望

针对芯片表面极其容易出现缺陷从而很难检测的这个问题，这篇论文去设计并且动手把一套建立在 C++ 和 Qt 框架基础上的工业视觉检测系统给做了出来；在软件的整体架构设计上，系统把上下文数据传输对象（DTO）跟信号槽多线程并发模型给融合到了一起，从而把工业级别处理过程当中必需的实时响应，以及数据流转的安全性给牢牢地保障住了；至于算法的具体实现，系统专门去搭建出了一条特别完整的视觉处理流水线，一方面，在前端把靠积分图来加速的单尺度 Retinex 光照均值化、图像金字塔还有快速 Sobel 算子这些方法综合运用了起来，以此来做到具有极强抗干扰能力的模板匹配，另一方面，在后端又把空间容错差分、自适应阈值分割以及靠并查集来提取连通域的技术给糅合到了一块，同时还会去结合长宽比和填充率等好几个维度的形态学特征，把专家决策树给成功地构建了出来；依靠这一系列的操作，系统成功地把缺陷的具体位置给精准找了出来，还能对像划痕、崩边或者异物这些毛病去进行定性的分类，并且完成了对全局质量的综合判断，把直观能看见的可视化结果给顺利呈现到了界面上。

不过因为受到了时间和实验条件上的限制，这个系统以后还能从下面这三个维度去进一步做深做透：第一点，就是要去优化底层的算力，可以把 SIMD 指令集或者是建立在 CUDA 上的 GPU 并行计算机制给引进来，通过这种方式把处理高分辨率图像时所消耗的算法时间给进一步压缩掉，向着极其苛刻的流水线检测节奏发起挑战；第二点，是在多维光学方面做进一步的拓展，可以去试着把多角度的结构光或者暗场照明结合进来，以此来弥补之前只用单一同轴光源带来的种种局限，从而把系统针对极其浅薄的划痕以及非常细微的坑洞去做三维特征反演的能力给大幅度提升上去；第三点，是要把人工智能的运作模式给引入进来，一点点地把依靠人工去死板调节阈值的做法给剥离掉，试着把算出的形态学特征喂给像 SVM 这样的机器学习模型，或者干脆直接引进像 YOLO 系列这种轻量级的深度学习网络，去把端到端的检测框架给重新构造一遍，进而赋予系统更为强大的泛化适应能力。

经过实际的实验验证可以看出，本项目不仅逻辑考虑得极其严密，跑起来也十分稳定，算是非常圆满地把针对芯片表面缺陷做精准检测和分类的这个预期目标给达成了；可以说系统在检测的精确度、抗干扰的鲁棒性以及实际执行的效率这几方面找到了一个极好的平衡点，这就让它具备了相当高的到工业现场去落地的潜力，展现出了极为切实的现实应用价值。

致谢

时光荏苒，岁月如梭，转眼间我的大学生活即将落下帷幕。随着这篇关于芯片视觉检测系统的论文顺利完稿，我的大学生涯也画上了一个圆满的句号。在此，我想向所有给予我帮助和支持的人表达最诚挚的谢意。

首先，我要特别感谢我的导师吕建勇老师。从最初的课题选定、系统架构设计，到后期算法模块的不断调试与论文的逐字修改，吕老师渊博的专业知识、严谨的治学态度和悉心的指导让我受益匪浅。这篇论文的顺利完成，离不开老师的心血与栽培。

其次，感谢计算机学院所有的授课老师们。是你们在课堂上为我打下了坚实的专业基础，让我有能力独立完成这样一项兼具理论与工程实践的综合性课题。

同时，我要感谢陪伴我度过无数个日夜的同学和朋友们。特别感谢孟维业、马韬、周瑞，在那些为了修复 Bug、跑通代码而焦头烂额的日子里，是你们的鼓励与探讨让我理清了思路，坚持到了最后。

最后，我要向我的家人致以最深的感恩。感谢你们二十多年来无条件的付出、包容与期盼，你们永远是我身后最坚实的后盾。

凡是过往，皆为序章。感谢这段充满挑战与成长的岁月，愿我们在未来的道路上各自努力，顶峰相见！

参考文献

- [1] N.G.Shankar,Z.W.Zhong.Defect detection on semiconductor wafer surfaces. Microelectronic Engineering,2005,77(3-4):337-346.
- [2] D.M.Tsai,C.C.Chang,S.M.Chao.Micro-crack inspection in heterogeneously textured solar wafers using anisotropic diffusion.Image and Vision Computing,2010,28(3):491-501.
- [3] Wang,J.;Fu,P.;Gao, R.X. Machine vision intelligence for product defect inspection based on deep learning and Hough transform. J. Manuf. Syst. 2019, 51:52–60.
- [4] 李可, 吴忠卿, 吉勇, 等. 改进 U-Net 芯片 X 线图像焊缝气泡缺陷检测方法[J]. 华中科技大学学报(自然科学版),2022,50(06):104-110.
- [5] 朱红艳, 李泽平, 赵勇, 等. 基于多尺度融合和可变形卷积 PCB 缺陷检测算法[J]. 计算机工程与设计, 2022,43(08):2188-2196.
- [6] 孙志超,王博,张晓玲.基于可变形残差卷积与伸缩式特征金字塔算法的 PCB 缺陷检测[J].电讯技术,2023,63(06):798-805.
- [7] 温悦欣. 基于机器视觉的 PCB 表面缺陷检测方法研究与系统实现[D]. 电子科技大学, 2020.
- [8] 张立国, 雷宣瑞, 金梅等. 基于图像处理的电路板缺陷检测系统设计[J]. 高新技术通讯. 2024,34(2):209-217.
- [9] 张定恒, 王守印, 叶世林, 等. 基于机器视觉的 PCB 裸板焊盘识别与提取方法[J]. 工业技术创新, 2023,10(05):26-34.
- [10] 巩玉奇, 陶晋宜, 杨刚. 基于改进中值滤波的手机玻璃瑕疵图像增强方法[J]. 电子技术应用, 2022,48(07):91-95.
- [11] 汪海涛, 林永铨, 陶维华, 等. 基于 GMR 传感器的国产伏羲芯片表面缺陷检测系统设计[J]. 计算机测量与控制, 2025,33(02):23-30+70.
- [12] 杨思念,曹立佳,杨洋,等.基于机器视觉的 PCB 缺陷检测算法研究综述[J].计算机科学与探索,2025,19(04):901-915.
- [13] Hanlin Y, Jun W. Automatic detection method of circuit boards defect based on Partition enhanced matching[J]. Information Technology Journal, 2013, 12(11): 2256.
- [14] 金贺楠.基于机器视觉的 SOP 芯片引脚缺陷检测的研究与应用[D].东北大学,2019.
- [15] 王平, 白秀玲. 基于改进模板匹配的芯片缺陷检测方法[J]. 微计算机信息 2007(01S):135-136.
- [16] 冯天桥, 毛征宇, 彭向前. 基于 FPGA 的运动目标实时检测系统设计[J]. 电子技术应用, 2023,49(06):99-103.

- [17] 阳斌, 谢亮, 金湘亮. 基于 FPGA 的实时图像采集与显示系统设计[J]. 中国集成电路, 2021,30(11):61-64+72.
- [18] 李庆春, 李祺, 刘彦君, 等. 基于 ZYNQ 的远程图像采集系统设计[J]. 计算机测量与控制, 2022,30(10):174-180.
- [19] 贺聪, 胡乃瑞, 李玉峰. 基于 FPGA 的自适应直方图均衡算法的研究与实现[J]. 电子设计工程, 2019,27(21):186-189.
- [20] 宁效龙, 何子力, 张昕昱, 等. 基于 Zynq 与 Qt 的视频采集与图像边缘检测系统[J]. 信息技术与网络安全, 2019,38(02):71-74+78.